

Міністерство освіти та науки України

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

ЗВІТ

з дисципліни: «Програмування веб-застосунків»

Практична робота №10

**Клієнтське програмування з бібліотеками та
фреймворками (Vue/React)**

Виконав:

Студент II курсу

групи ІІІ-56

Гончар І.В.

Перевірив:

Ковтунець О.В.

1. Мета роботи:

Обрати клієнтський фреймворк Vue 3 та підключити його через CDN без використання складених збірок чи Node.js. Перенести список елементів нашого варіанта (каталог настільних ігор), що раніше рендерився вручну через DOM API, на компонентний підхід: окремий компонент GameCard отримує дані через props і рендерить їх декларативно за допомогою директиви v-for. Реалізувати інтерактивну подію фільтрації за жанром через механізм \$emit, яка автоматично оновлює реактивний стан і перемальовує інтерфейс без жодного ручного використання querySelector чи innerHTML.

2. Хід виконання роботи

Крок 1-2. Вибір фреймворку та підключення CDN:

Для виконання роботи було обрано фреймворк **Vue 3**, оскільки він забезпечує зручний декларативний рендеринг і реактивність через глобальну CDN-збірку без необхідності налаштування складних середовищ збірки. До HTML-розмітки підключено бібліотеку Vue 3 за допомогою тега `<script>`.

Крок 3-5. Перенесення даних у реактивний стан та створення компонентів:

Масив об'єктів ігор перенесено у реактивний стан (`data()`) кореневого компонента. Створено окремий дочірній компонент GameCard, який приймає через props поля `title`, `minPlayers`, `maxPlayers` та `genre`, а також рендерить їх за допомогою директиви `v-for` із обов'язковим унікальним атрибутом `:key` (рис. 1,2).

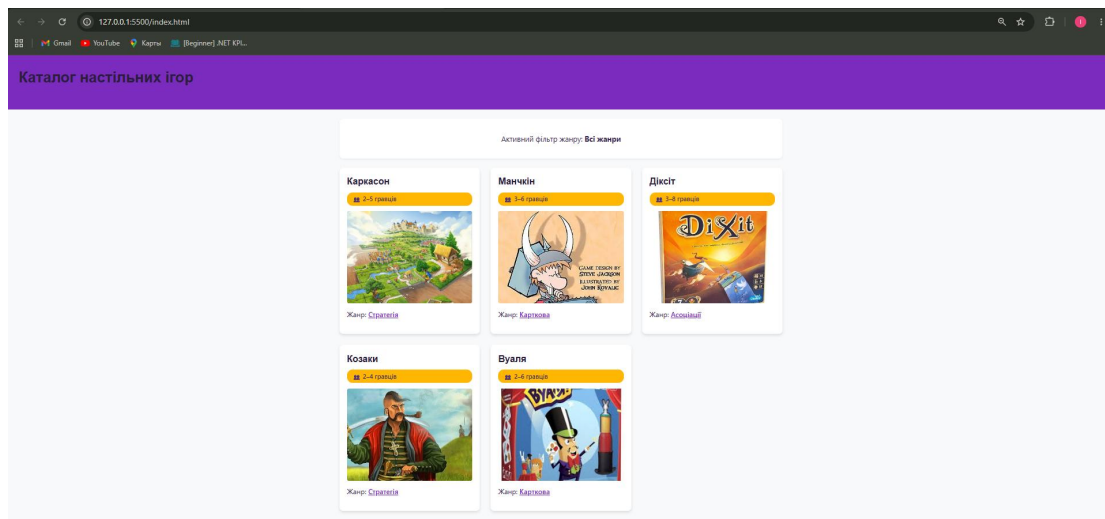


Рис. 1 — Загальний вигляд каталогу настільних ігор на Vue 3 (Google Chrome)

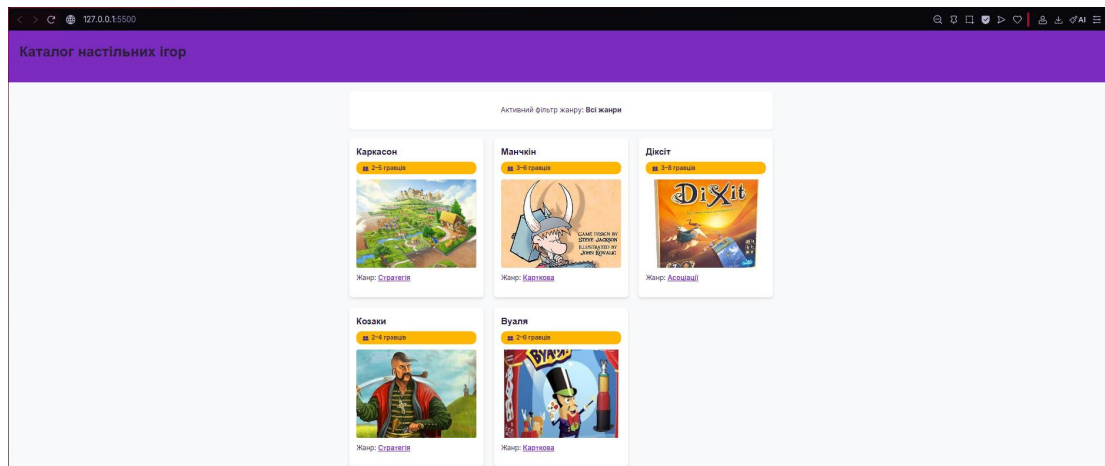


Рис. 2 — Загальний вигляд каталогу настільних ігор на Vue 3 (Opera GX)

Крок 6-8. Реалізація інтерактивної події та реактивності:

Налаштовано обробку кліку по жанру в картці гри, який за допомогою події `$emit('filter', genre)` передає значення вгору до батьківського компонента. Завдяки обчислюваній властивості (`computed`) список ігор фільтрується та оновлюється автоматично одразу при зміні стану `currentGenre`, повністю виключаючи маніпуляції з ручним DOM-деревом (рис. 3).

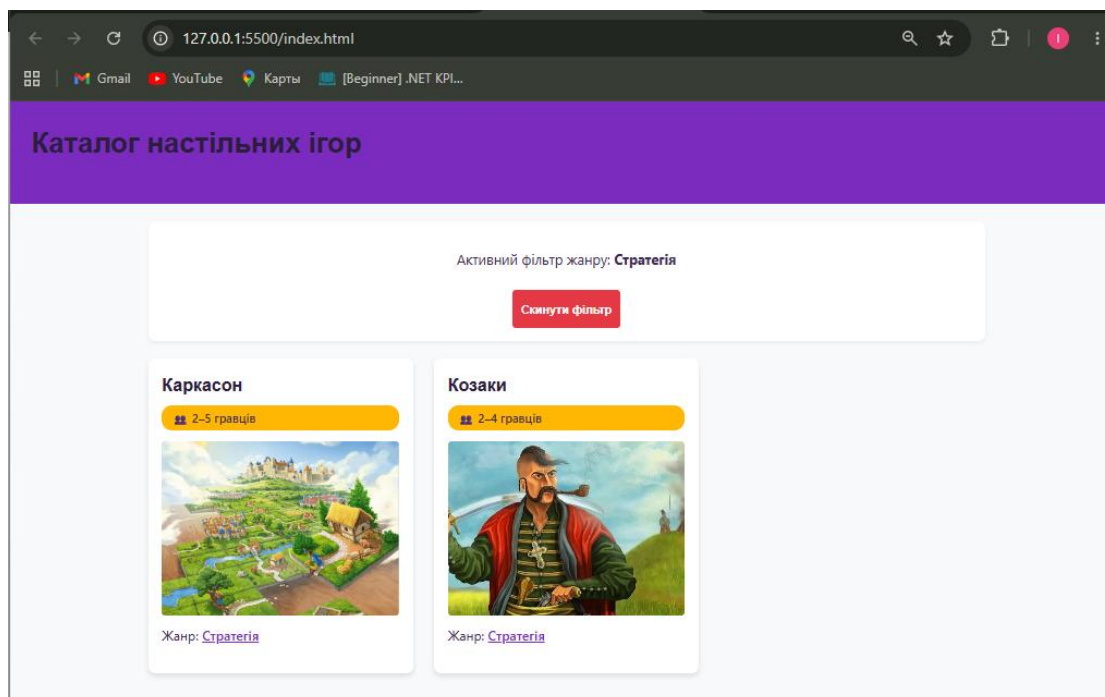


Рис. 3 — Результат роботи реактивного фільтра за жанром гри

Крок 9. Перевірка у DevTools:

За допомогою розширення браузера **Vue DevTools** перевірено коректність побудови дерева компонентів, передачу пропсів (props) у GameCard та поточний стан реактивних змінних у реальному часі (рис. 4).

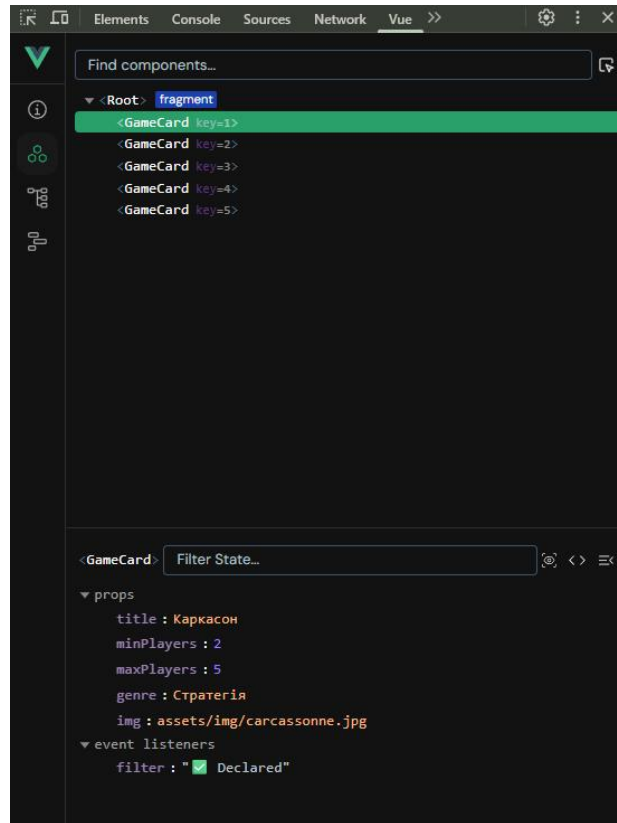


Рис. 4 — Перевірка компонентів і поточного стану у Vue DevTools

Крок 10. Версіонування та збереження:

Усі оновлення коду зафіксовано системою контролю версій Git з описовим повідомленням та відправлено до віддаленого репозиторію на платформі GitHub (рис. 5).

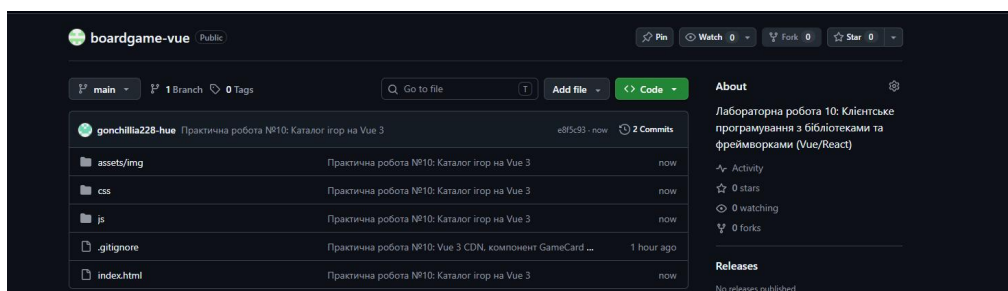


Рис. 5 — Фіксація змін у системі контролю версій Git на платформі GitHub

Посилання на репозиторій:

<https://github.com/Bitshkiper218/boardgame-vue>

Висновок:

Під час виконання практичної роботи було успішно освоєно базові принципи компонентного підходу та реактивності з використанням клієнтського фреймворку Vue 3 (через CDN). Застарілий імперативний підхід до маніпуляції DOM було успішно замінено на сучасний декларативний: створено ізольований перевикористовуваний компонент GameCard, налаштовано передачу даних до нього через props та реалізовано відображення списку за допомогою директиви v-for. Отримано практичний досвід налаштування зв'язку між дочірнім та батьківським компонентами за допомогою користувацьких подій (\$emit), а також створення реактивних обчислюваних властивостей (computed). Завдяки цьому було реалізовано миттєву фільтрацію каталогу настільних ігор за жанром без жодного ручного втручання в DOM-дерево. Коректність роботи стану та компонентів перевірено за допомогою Vue DevTools, а фінальний код проєкту успішно версіоновано та опубліковано на платформі GitHub.